

EXPERIMENT-4

1. Create a Book class with attributes title and author.

Assign default values (e.g., "Untitled", "Unknown Author").

Accept user-defined values for title and author.

Add another constructor that also includes an integer parameter for publicationYear.

Objective: Create multiple Book objects using the different constructors and display their details.

Book.java	Run	Output
<pre>1- public class Book { 2 private String title; 3 private String author; 4 private int publicationYear; 5 6 // Default constructor 7- public Book() { 8 this.title = "Untitled"; 9 this.author = "Unknown Author"; 10 this.publicationYear = 0; 11 } 12 13 // Constructor with title and author 14- public Book(String title, String author) { 15 this.title = title; 16 this.author = author; 17 this.publicationYear = 0; 18 } 19 20 // Constructor with title, author, and publicationYear 21- public Book(String title, String author, int publicationYear) { 22 this.title = title; 23 this.author = author; 24 this.publicationYear = publicationYear; 25 } 26</pre>		<pre>Title: Untitled Author: Unknown Author Publication Year: N/A Title: 1984 Author: George Orwell Publication Year: N/A Title: To Kill a Mockingbird Author: Harper Lee Publication Year: 1960 === Code Execution Successful ===</pre>

```
Book.java
25     }
26
27     // Method to display book details
28     public void displayDetails() {
29         System.out.println("Title: " + title);
30         System.out.println("Author: " + author);
31         System.out.println("Publication Year: " + (publicationYear != 0 ? publicationYear : "N/A"));
32     }
33
34     public static void main(String[] args) {
35         // Creating book objects using different constructors
36         Book book1 = new Book();
37         Book book2 = new Book("1984", "George Orwell");
38         Book book3 = new Book("To Kill a Mockingbird", "Harper Lee",
39                               1960);
40
41         // Displaying details of each book
42         book1.displayDetails();
43         System.out.println();
44         book2.displayDetails();
45         System.out.println();
46         book3.displayDetails();
47     }
48 }
```

2. Declare a Customer class with a private variable balance.

Implement a public addBalance(double amount) method to add funds.

Implement a protected deductBalance(double amount) method to reduce funds.

Define a default-access (package-private) method showBalance() to display the current balance.

Overload the addBalance(...) method to also accept an int amount for smaller deposits.

Objective: Create an instance of the Customer class Showcase method overloading for different deposit parameters.

Customer.java	Run	Output
<pre>1 public class Customer { 2 private double balance; 3 public void addBalance(double amount) { 4 if (amount > 0) { 5 balance += amount; 6 } 7 } 8 public void addBalance(int amount) { 9 if (amount > 0) { 10 balance += amount; 11 } 12 } 13 protected void deductBalance(double amount) { 14 if (amount > 0 && amount <= balance) { 15 balance -= amount; 16 } 17 } 18 void showBalance() { 19 System.out.println("Current balance: " + balance); 20 } 21 public static void main(String[] args) { 22 Customer customer = new Customer(); 23 customer.addBalance(100.50); 24 customer.addBalance(50); 25 customer.showBalance(); 26 customer.deductBalance(30); 27 } 28 }</pre>	Run	<pre>Current balance: 150.5 Current balance: 120.5 === Code Execution Successful ===</pre>

Customer.java	Run
<pre>5 balance += amount; 6 } 7 } 8 public void addBalance(int amount) { 9 if (amount > 0) { 10 balance += amount; 11 } 12 } 13 protected void deductBalance(double amount) { 14 if (amount > 0 && amount <= balance) { 15 balance -= amount; 16 } 17 } 18 void showBalance() { 19 System.out.println("Current balance: " + balance); 20 } 21 public static void main(String[] args) { 22 Customer customer = new Customer(); 23 customer.addBalance(100.50); 24 customer.addBalance(50); 25 customer.showBalance(); 26 customer.deductBalance(30); 27 customer.showBalance(); 28 } 29 }</pre>	Run

3. Create a Club class with a static variable clubName and a non-static variable memberName.

Provide a static method displayClubName() to print the club's name. Create multiple Club objects and assign different member names to each.

Objective: Show how the static variable clubName is shared across all instances, while memberName remains unique to each object. Prove this by displaying each member's name alongside the shared club name.

Product.java	Output
<pre>1 public class Product { 2 private int productId; 3 private String productName; 4 private String category; 5 private double price; 6 private static int totalProducts = 0; 7 public Product() { 8 this.productId = 0; 9 this.productName = "Default Product"; 10 this.category = "Default Category"; 11 this.price = 0.0; 12 totalProducts++; 13 } 14 public Product(int productId, String productName, String category, double price) { 15 this.productId = productId; 16 this.productName = productName; 17 this.category = category; 18 this.price = price; 19 totalProducts++; 20 } 21 public double getPrice() { 22 return price; 23 } 24 public void displayProductInfo() { 25 System.out.println("Product ID: " + productId);</pre>	<pre>Total Products: 3 Product ID: 0 Product Name: Default Product Category: Default Category Price: 0.0 Stock Value: 0.0 Product ID: 1 Product Name: Laptop Category: Electronics Price: 1000.0 Stock Value: 5000.0 Stock Value with Discount: 4500.0 Product ID: 2 Product Name: Smartphone Category: Electronics Price: 500.0 Stock Value: 10000.0 Stock Value with Discount: 8500.0 === Code Execution Successful ===</pre>

```
Product.java    Share  Run
26     System.out.println("Product Name: " + productName);
27     System.out.println("Category: " + category);
28     System.out.println("Price: " + price);
29 }
30 public static void displayTotalProducts() {
31     System.out.println("Total Products: " + totalProducts);
32 }
33 public double calculateStockValue(int quantity) {
34     return price * quantity;
35 }
36 public double calculateStockValue(int quantity, double
    discountRate) {
37     return price * quantity * (1 - discountRate);
38 }
39 public static void main(String[] args) {
40     Product product1 = new Product();
41     Product product2 = new Product(1, "Laptop", "Electronics",
        1000.0);
42     Product product3 = new Product(2, "Smartphone",
        "Electronics", 500.0);
43     displayTotalProducts();
44     product1.displayProductInfo();
45     System.out.println("Stock Value: " + product1
        .calculateStockValue(10));
46     System.out.println();
47     product2.displayProductInfo();
```

```
Product.java    Share  Run
37     return price * quantity * (1 - discountRate);
38 }
39 public static void main(String[] args) {
40     Product product1 = new Product();
41     Product product2 = new Product(1, "Laptop", "Electronics",
        1000.0);
42     Product product3 = new Product(2, "Smartphone",
        "Electronics", 500.0);
43     displayTotalProducts();
44     product1.displayProductInfo();
45     System.out.println("Stock Value: " + product1
        .calculateStockValue(10));
46     System.out.println();
47     product2.displayProductInfo();
48     System.out.println("Stock Value: " + product2
        .calculateStockValue(5));
49     System.out.println("Stock Value with Discount: " + product2
        .calculateStockValue(5, 0.1));
50     System.out.println();
51     product3.displayProductInfo();
52     System.out.println("Stock Value: " + product3
        .calculateStockValue(20));
53     System.out.println("Stock Value with Discount: " + product3
        .calculateStockValue(20, 0.15));
54 }
55 }
```

4. Define a Car class with attributes make, model, and year.

Create a parameterized constructor that uses the this keyword to distinguish constructor parameters from the class fields (e.g., this.make = make;). Instantiate a Car object with specific values (e.g., "Tesla", "Model 3", 2025) and display its details, ensuring the correct assignment of attributes using this.

Car.java	Output
<pre>1 public class Car { 2 private String make; 3 private String model; 4 private int year; 5 6 public Car(String make, String model, int year) { 7 this.make = make; 8 this.model = model; 9 this.year = year; 10 } 11 12 public void displayDetails() { 13 System.out.println("Make: " + make); 14 System.out.println("Model: " + model); 15 System.out.println("Year: " + year); 16 } 17 18 public static void main(String[] args) { 19 Car car = new Car("Tata", "Model 1", 2025); 20 car.displayDetails(); 21 } 22 }</pre>	<pre>Make: Tata Model: Model 1 Year: 2025 === Code Execution Su</pre>

5. A company wants to manage a list of products with details such as product ID, name, price, and category. They also want to keep track of the total number of products and calculate the overall stock value.

Product Class:

Instance variables: productId, productName, category.

A private variable price.

A default constructor setting some default values.

A parameterized constructor that initializes product details using this to differentiate parameters from class fields.

A public method getPrice() to access the private price.

A public method displayProductInfo() to show all details.

Static Members:

A static variable totalProducts to count how many Product objects are created.

A static method displayTotalProducts() to print the total count.

Method to Calculate Stock Value:

A method (e.g., calculateStockValue()) to multiply the price by a given quantity (demonstrate method overloading by adding an optional parameter for a discount rate).

In the main method:

Create multiple Product objects using both the default and parameterized constructors.

Call the static method to display the total number of products.

Display each product's details, including the price and the calculated stock value for a given quantity.

Club.java	Output
<pre>1 public class Club { 2 private static String clubName; 3 private String memberName; 4 public Club(String memberName) { 5 this.memberName = memberName; 6 } 7 public static void setClubName(String name) { 8 clubName = name; 9 } 10 public static void displayClubName() { 11 System.out.println("Club Name: " + clubName); 12 } 13 public void displayMemberInfo() { 14 System.out.println("Member Name: " + memberName + ", Club 15 Name: " + clubName); 16 } 17 public static void main(String[] args) { 18 Club.setClubName("Chess Club"); 19 Club member1 = new Club("Alice"); 20 Club member2 = new Club("Bob"); 21 Club member3 = new Club("Charlie"); 22 Club.displayClubName(); 23 member1.displayMemberInfo(); 24 member2.displayMemberInfo(); 25 member3.displayMemberInfo(); 26 } 27 }</pre>	<pre>Total Products: 3 Product ID: 0 Product Name: Default Product Category: Default Category Price: 0.0 Stock Value: 0.0 Product ID: 1 Product Name: Laptop Category: Electronics Price: 1000.0 Stock Value: 5000.0 Stock Value with Discount: 4500.0 Product ID: 2 Product Name: Smartphone Category: Electronics Price: 500.0 Stock Value: 10000.0 Stock Value with Discount: 8500.0 === Code Execution Successful ===</pre>